

数据库系统原理实验—实验九

ER图的绘制

学　　号:23336179

姓　　名:马福泉

专　　业:计算机科学与技术

课 程 教 师:赖韩江

起 始 日 期:2025年11月20日

结 束 日 期:2025年11月27日

实 验 地 点:B202

目录

1 实验概述 **2**

2 实验报告 **2**

2.1 电影院数据库设计 2

2.1.1 E-R图 2

2.1.2 实体（Entity）的设计与属性 3

2.1.3 关系（Relationship）及其约束 4

2.1.4 关系模型转换 5

2.2 School 数据库设计 6

2.2.1 E-R图 6

2.2.2 E-R图设计说明 6

3 实验总结 **8**

3.1 实验难点与注意事项 8

3.2 实验收获 8

3.3 总结图例规范 9

1 实验概述

1. 数据库设计

为某电影院设计数据库。每部电影需记录名字、主演、导演、发行公司信息。一部电影上映多个场次，场次要记录播放时间，影厅号，影厅类型（IMAX，杜比、普通厅等）、原票价。每个场次售出多张电影票，电影票要记录座位号、实际票价。影院设有会员制，一个会员可购买多张电影票，会员要记录会员卡号，姓名，联系方式，充值记录。

（1）根据上述语义画出E-R图。

（2）将该E-R模型转换为关系模型。

2. 为实验教材上的 school 数据库画E-R图。

2 实验报告

2.1 电影院数据库设计

2.1.1 E-R图

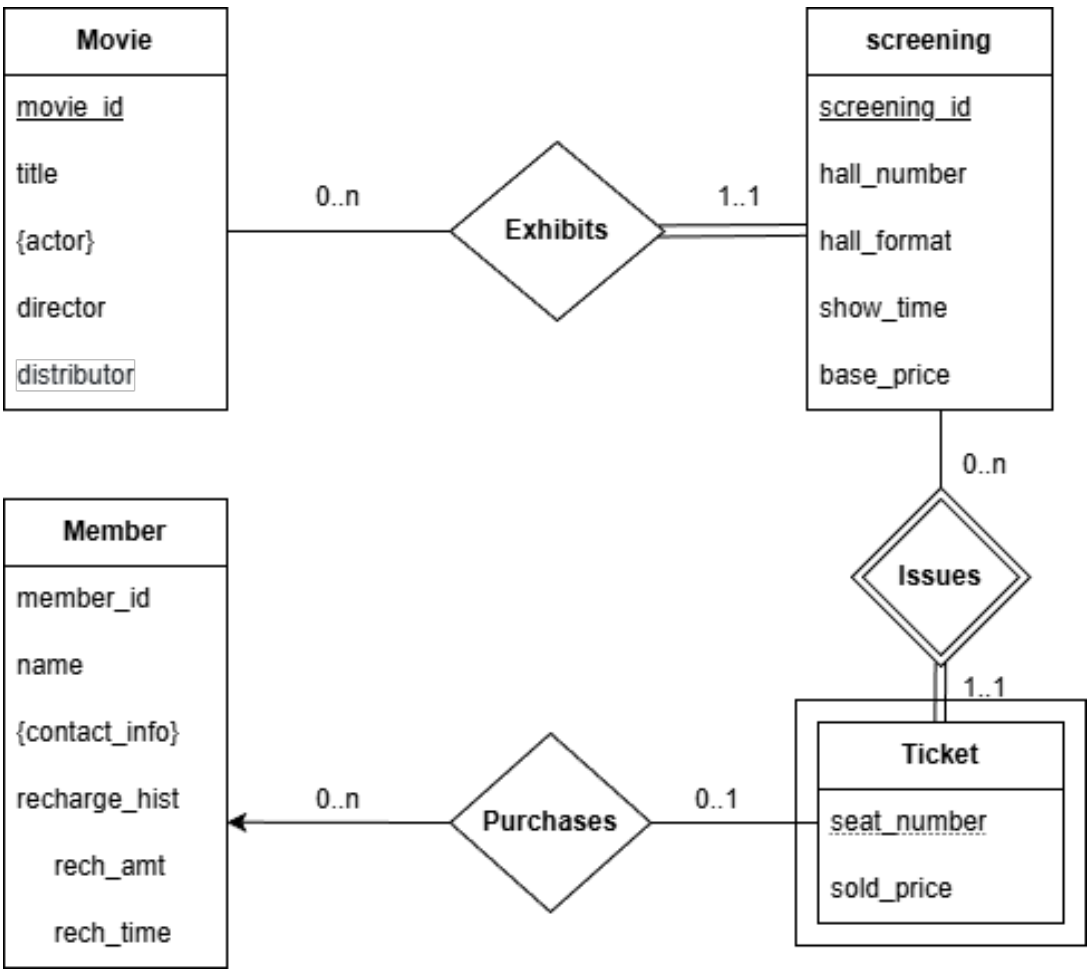


图 1: 电影院数据库E-R图

2.1.2 实体（Entity）的设计与属性

图中包含四个主要实体，分别对应现实世界中的四个核心对象：

1. **Movie（电影）：**
   - **性质：** 强实体。
   - **属性：** 选取 `movie_id` 作为主键（Primary Key，下划线标出）唯一标识一部电影。`title`（电影名）、`director`（导演）、`distributor`（发行公司）为普通单值属性。
   - **特殊属性：** {`actor`} 被设计为多值属性（Multivalued Attribute），用花括号 {} 标记。这对应需求中“一部电影有多个主演”的语义。
2. **Screening（场次）：**
   - **性质：** 强实体（引入代理键优化）。
   - **属性：** `screening_id`（主键）、`show_time`、`hall_number`、`hall_format`、`base_price`。
   - **设计说明（为什么要加 ID?）：**
     - **主键臃肿问题：** 如果 Screening 是弱实体，它的主键将是复合主键（如 Movie_ID + Show_Time + Hall_Number）。
     - **连锁反应：** Ticket 也是弱实体，依赖于 Screening。如果 Screening 没有独立 ID，Ticket 的主键将变成：Movie_ID + Show_Time + Hall_Number + Seat_Number。一张电影票表的主键包含4个字段，会导致查询慢、索引大、外键关联麻烦。
     - **引入代理键 (Surrogate Key)：** 因此，“人为”引入了 `screening_id` 作为代理主键，使其成为强实体，便于管理和引用。
3. **Member（会员）：**
   - **性质：** 强实体。
   - **属性：** `member_id` 为主键。{`contact_info`} 为多值属性，表示会员可能有多个联系方式。
   - **复合属性：** `recharge_hist`（充值记录）包含 `rech_amt`（金额）和 `rech_time`（时间）。在概念模型中，这表示会员包含充值历史信息。
4. **Ticket（电影票）：**
   - **性质：** 弱实体（Weak Entity）。
   - **画法解释：** 使用双边框矩形表示。
   - **理由：** 电影票的“座位号”在整个影院系统中不是唯一的（不同场次都有1排1座）。电影票必须依赖于具体的“场次”才能存在。因此，它没有完整的主键。
   - **属性：** `seat_number` 下方画了虚线，表示它是部分键（Partial Key）或分辨符。它只能在某一个特定的场次中区分不同的票。

2.1.3 关系（Relationship）及其约束

图中通过菱形框表示实体间的联系，具体的基数比（Cardinality）和参与度（Participation）解释如下：

1. **Exhibits（放映）关系**
   - **连接实体：** Movie 与 Screening
   - **语义：** 一部电影被安排在多个场次放映。
   - **基数约束 (0..n : 1..1)：**
     - **Movie端 (0..n)：** 一部电影可以对应 0 个或 n 个场次（一对多关系）。0 表示电影已录入系统但尚未排片。
     - **Screening端 (1..1)：** 一个场次必须且只能放映一部电影。
   - **双线（Double Line）含义：** Screening 侧使用了双线连接菱形，表示全参与（Total Participation）。即：不存在“没有电影”的空白场次，任何一个场次实体的存在，都必须依附于一部电影。
2. **Issues（出票/包含）关系—— 弱实体联系**
   - **连接实体：** Screening 与 Ticket
   - **画法解释：** 使用双边框菱形。
   - **含义：** 这是识别性关系（Identifying Relationship）。它专门用于连接弱实体（Ticket）和它的标识实体（Screening）。弱实体必须通过这个关系来确定身份。
   - **基数约束 (0..n : 1..1)：**
     - **Screening端 (0..n)：** 一个场次包含多张电影票（座位）。
     - **Ticket端 (1..1)：** 一张电影票必须归属于一个特定的场次。
   - **双线含义：** Ticket 侧为双线，表示 Ticket 对该关系是全参与的。一张票脱离了场次就没有意义。
3. **Purchases（购买）关系**
   - **连接实体：** Member 与 Ticket
   - **语义：** 会员购买电影票。
   - **基数约束 (0..n : 0..1)：**
     - **Member端 (0..n)：** 需求指出“一个会员可购买多张电影票”，所以是 n。0 表示会员注册后可以暂时不买票。
     - **Ticket端 (0..1)：** 一张具体的电影票，或者属于某一个会员（1），或者不属于任何会员（0）。
   - **关于0..1的解释：** 这里的 0 涵盖了两种情况：一是票还未售出；二是票被非会员（散客）购买，未关联到会员ID。这符合实际业务逻辑。

2.1.4 关系模型转换

将 E-R 图转换为关系模型（即数据库表结构），遵循以下规则：

- **强实体**转为独立的表。
- **弱实体**转为独立的表，但必须包含父实体的主键作为外键。
- **1:n 关系**：将“1”端的主键加入到“n”端的表中作为外键。
- **多值属性**（如 {actor}）：必须单独拆分成一张新表。

基于上述规则，转换得到的最终关系模式如下（如图 2 所示）：

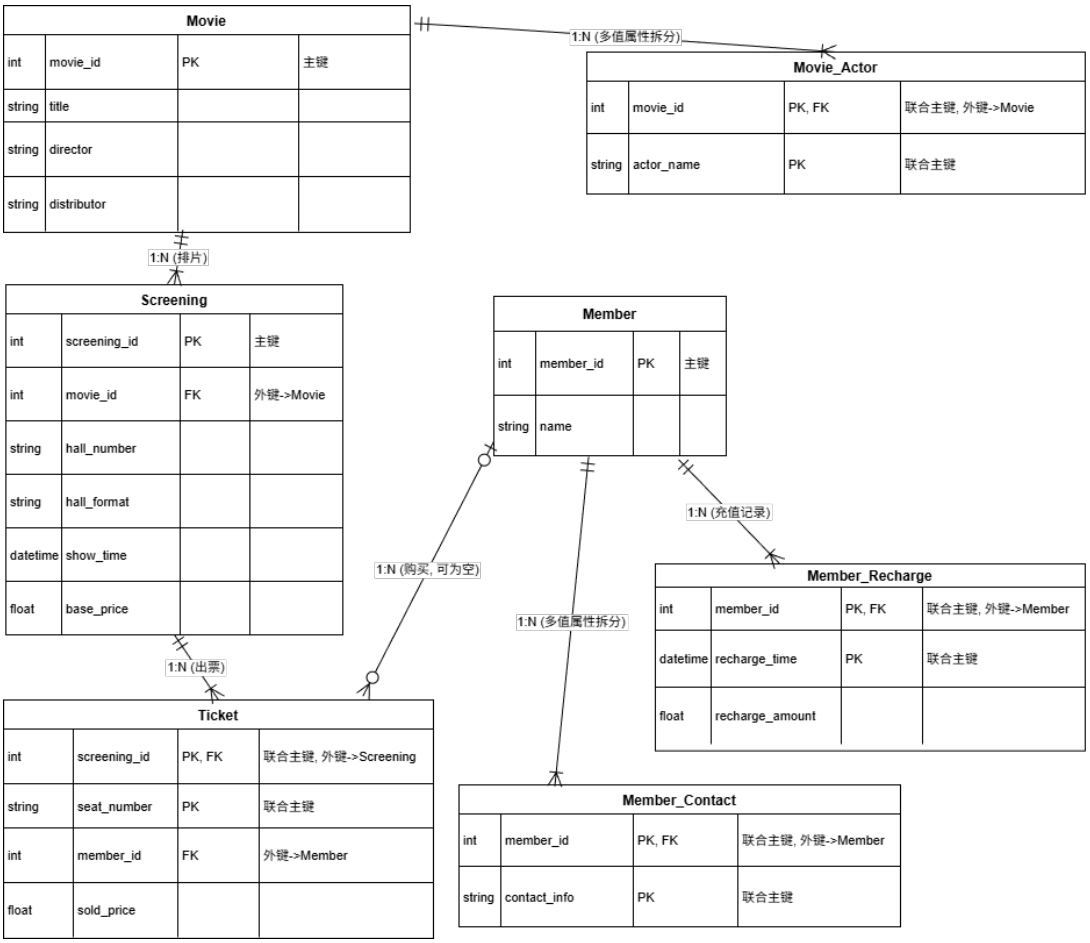


图 2: 关系模型转换示意图

1. 基础实体转换

- **Movie (电影表)**: Movie(<u>movie_id</u>, title, director, distributor)
- **Member (会员表)**: Member(<u>member_id</u>, name)

2. 处理多值/复合属性（拆分为新表）

- **Movie_Actor (电影主演表)**: Movie_Actor(<u>movie_id, actor_name</u>)
  外键：*movie_id* 引用 Movie(movie_id)

- **Member_Contact (会员联系方式表)**: Member_Contact(member_id, contact_info)
  外键：*member_id* 引用 Member(member_id)

- **Member_Recharge (会员充值记录表)**: Member_Recharge(member_id, recharge_time, recharge_amount)
  外键：*member_id* 引用 Member(member_id)

3. 处理关系与外键

- **Screening (场次表)**: Screening(screening_id, hall_number, hall_format, show_time, base_price, *movie_id*)
  外键：*movie_id* 引用 Movie(movie_id)

- **Ticket (电影票表)**: Ticket(screening_id, seat_number, sold_price, *member_id*)
  外键：*screening_id* 引用 Screening(screening_id)
  外键：*member_id* 引用 Member(member_id)

2.2 School 数据库设计

2.2.1 E-R图

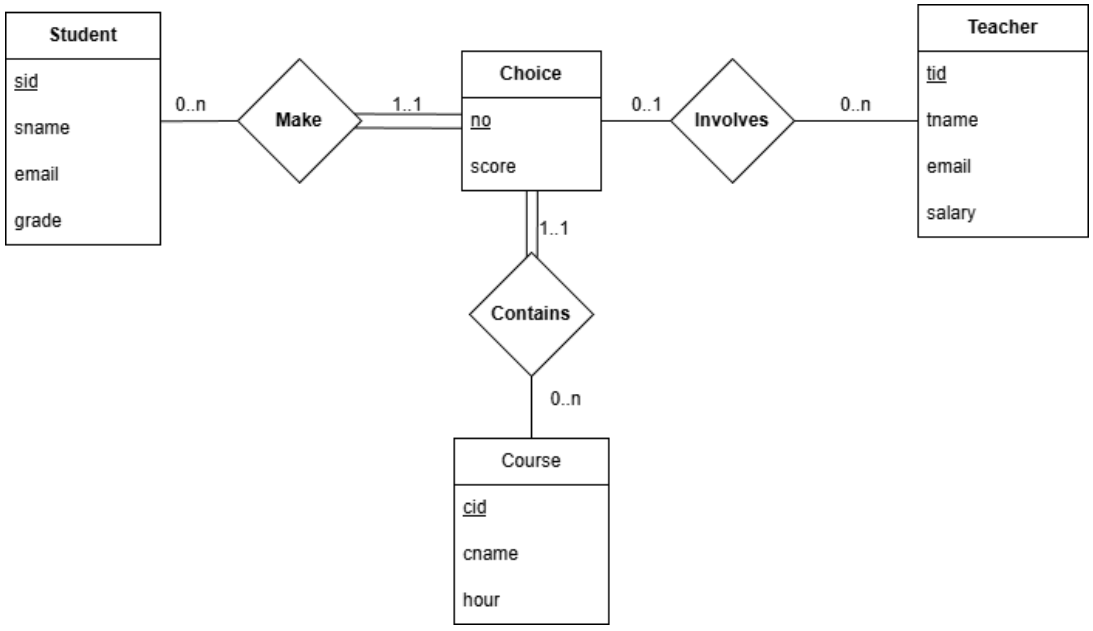


图 3: School 数据库 E-R 图

2.2.2 E-R图设计说明

以下内容为系统概念模型设计说明。

A. 实体 (Entities) 的定义 本系统包含四个强实体（Strong Entities），它们在图中均用普通矩形框表示，因为它们都拥有独立的、唯一的主键标识。

- **Student (学生):** 主键为 `sid`，拥有属性 `sname` (姓名), `email` (邮箱), `grade` (年级)。

- **Teacher (教师):** 主键为 `tid`，拥有属性 `tname` (姓名), `email` (邮箱), `salary` (薪水)。

- **Course (课程):** 主键为 `cid`，拥有属性 `cname` (课程名), `hour` (学时)。
- **Choice (选课记录):** 虽然在逻辑上它关联了学生、教师和课程，但根据数据库定义，它拥有独立的流水号主键 `no`，因此将其建模为强实体（关联实体）。属性包括 `score` (成绩)。

B. 关系 (Relationships) 与基数 (Cardinality) 解释 图中将传统的“学生-课程-教师”三元多对多关系，拆解为三个二元的一对多关系，中心围绕 Choice 实体展开。

1. **Make 关系 (Student —— Choice)**
   - **语义：** 学生进行选课，产生选课记录。
   - **基数约束 (0..n : 1..1)：**
     - **Student 端 (0..n)：** 一个学生可以有多条选课记录，也可以暂时不选课 (0)。
     - **Choice 端 (1..1)：** 一条选课记录必须且只能属于一个特定的学生。
   - **参与度 (双实线/单实线)：** Choice 侧为双实线（全参与）：根据 SQL 定义 `[sid] [char](10) NOT NULL`，每一条选课记录必须关联有效的学生 ID，不能为空。
2. **Contains 关系 (Course —— Choice)**
   - **语义：** 选课记录包含特定的课程。
   - **基数约束 (0..n : 1..1)：**
     - **Course 端 (0..n)：** 一门课程可以被选多次，生成多条记录，也可以没人选 (0)。
     - **Choice 端 (1..1)：** 一条选课记录对应且仅对应一门课程。
   - **参与度 (双实线/单实线)：** Choice 侧应为双实线（全参与）：根据 SQL 定义 `[cid] [char](10) NOT NULL`，选课记录必须包含课程信息。
3. **Involves 关系 (Teacher —— Choice)**
   - **语义：** 选课记录涉及特定的授课教师。
   - **基数约束 (0..n : 0..1)：**
     - **Teacher 端 (0..n)：** 一个老师可以讲授多条选课记录所对应的课。
     - **Choice 端 (0..1)：** 注意这里是 0..1。
     - **理由：** 根据 SQL 定义 `[tid] [char](10) NULL` (允许为空)。这意味着一条选课记录可能暂时没有指定老师（例如课程已选，老师待定）。
   - **参与度 (单实线)：** 两侧均为单实线（部分参与）：Choice 实体并非必须关联老师，体现了数据库中外键可为空的特性。

3. 总结 该 E-R 图较为准确反映了提供的 SQL 架构，需要注意的地方：

- 识别出 CHOICES 表因具备独立主键 `no` 而成为强实体。
- 准确表达了 `sid` 和 `cid` 的 NOT NULL 约束（表现为全参与/双实线）。
- 准确表达了 `tid` 的 NULL 约束（表现为 0..1 和单实线）。

3 实验总结

3.1 实验难点与注意事项

在本次实验中，有几个地方容易混淆，需要注意：

1. **E-R 图中不应包含外键**
   - **误区：** 我常习惯在 Screening 实体中画上 `movie_id` 属性。
   - **正解：** 在概念模型（E-R图）中，实体间的联系（Relationship）本身就承担了引用的功能。如果在实体中再写外键，属于信息冗余。
   - **转换：** 只有在将 E-R 图转换为关系模式（逻辑设计）时，才需要将 1:n 关系中的“1”端主键作为外键加入到“n”端表中。
2. **弱实体（Weak Entity）的识别与表示**
   - **识别：** 本实验中的 Ticket 是典型的弱实体。它的主键（座位号）在全局不唯一，必须依赖于 Screening（场次）才能确定身份。
   - **表示：** 需使用双边框矩形表示实体，使用双边框菱形表示其与强实体的识别性关系。
   - **参与度：** 弱实体对其识别性关系通常是全参与的（双实线），因为脱离了强实体，弱实体就没有存在的意义。
3. **多值属性的处理**
   - **设计：** 如 Movie 的 {`actor`} 和 Member 的 {`contact_info`}。在 E-R 图中需明确标记（如花括号）。
   - **实现：** 在转换为关系模式时，切记多值属性不能直接作为一个字段存储（违背第一范式），必须将其拆分为独立的关联表（如 Movie_Actor 表）。

3.2 实验收获

通过本次实验，我获得了以下收获：

1. **掌握了从需求到模型的转化方法：** 学会了如何从复杂的文字需求中提取实体、属性和关系，并判断实体间的基数比（1:1, 1:n, m:n）。
2. **深入理解了概念模型与逻辑模型的区别：** 明确了 E-R 图关注业务逻辑和事物联系，而关系模式关注数据在计算机中的存储结构。
3. **强化了数据库完整性约束的理解：** 通过 School 数据库的案例，初步理解了 SQL 定义中的 `NOT NULL` 约束对应 E-R 图中的全参与（双线），而 `NULL` 对应部分参与（单线）。
4. **提升了规范化文档编写能力：** 使用 LaTeX 撰写包含图表、公式和规范化图例说明的数据库设计报告。

3.3 总结图例规范

以下是本次实验体现的的 E-R 图图例规范：

- **矩形框：** 表示强实体（如 Movie）。
- **双矩形框：** 表示弱实体（Ticket），依赖于强实体存在。
- **椭圆/列表属性：** 实体的特征。
- **下划线属性：** 主键（Primary Key）。
- **虚下划线属性：** 弱实体的部分键（Partial Key）。
- **花括号 {}：** 表示多值属性。
- **菱形框：** 表示实体间的关系。
- **双菱形框：** 表示弱实体的识别性关系。
- **连线上的数字 (min..max)：** 表示基数约束（Cardinality Constraints），定义了数量对应关系。
- **双横线：** 表示全参与（该实体在关系中必须存在）。